🛒 E-Commerce Sales Analytics

End-to-End SQL Case Study & Business Intelligence Documentation

MS SQL SERVER | T-SQL | WINDOW FUNCTIONS | BUSINESS INSIGHTS

📌 Project Overview

This document provides an end-to-end exploratory and analytical data investigation into an **E-Commerce Sales Dataset**. Using **Microsoft SQL Server (T-SQL)**, this case study addresses critical business questions ranging from foundational operational metrics to complex financial trends, regional performance, and customer satisfaction analysis.

🗄️ Database Schema

Target Table: `dbo.ECommerce_Sales_Analytics`

COLUMN NAME	DATA TYPE	DESCRIPTION
`order_id`	INT / VARCHAR	Unique identifier for each customer transaction
`order_date`	DATE / DATETIME	Date on which the order was placed
`customer_id`	INT / VARCHAR	Unique identifier for each customer
`product_category`	VARCHAR	Category of the product purchased
`region`	VARCHAR	Geographical region where the order was placed
`quantity`	INT	Number of units purchased
`unit_price`	DECIMAL	Price per single unit
`discount`	DECIMAL	Discount applied to the order (e.g., 0.10 = 10%)
`payment_method`	VARCHAR	Payment mode used (Credit Card, Wallet, COD, etc.)
`delivery_days`	INT	Time taken (in days) to deliver the order
`customer_rating`	DECIMAL	Rating given by the customer (1 to 5 scale)
`revenue`	DECIMAL	Net revenue generated from the order

🚀 Key SQL Techniques Demonstrated

- **Basic Aggregations:** `COUNT()`, `SUM()`, `AVG()`, `MAX()`, `MIN()`
- **Data Categorization:** Multi-level conditional logic using `CASE WHEN`
- **Subqueries & CTEs:** Common Table Expressions for query modularity and reusability
- **Window Functions:** `DENSE_RANK() OVER (PARTITION BY ...)`, `SUM() OVER(ORDER BY ...)`
- **Date Manipulation:** `YEAR()`, `MONTH()` for temporal trend analysis

📜 Case Study Questions, SQL Queries & Solutions

🟢 SECTION 1: Basic Level (Foundational Metrics)

1. Total Order Count

Retrieve the total number of orders placed in the dataset.

```
SELECT COUNT(order_id) AS total_orders
FROM dbo.ECommerce_Sales_Analytics;
```

2. Total Business Revenue

Calculate the total revenue generated from all sales.

```
SELECT ROUND(SUM(revenue), 2) AS total_revenue
FROM dbo.ECommerce_Sales_Analytics;
```

3. Highest Priced Product Item

Identify the highest unit price item sold across all orders.

```
SELECT MAX(unit_price) AS max_unit_price
FROM dbo.ECommerce_Sales_Analytics;
```

4. Average Customer Rating

Find the average customer rating across all orders.

```
SELECT ROUND(AVG(customer_rating), 2) AS avg_customer_rating
FROM dbo.ECommerce_Sales_Analytics;
```

5. Total Quantity Sold by Product Category

List the total quantity sold for each product category.

```
SELECT
    product_category,
    SUM(quantity) AS total_quantity_sold
FROM dbo.ECommerce_Sales_Analytics
GROUP BY product_category
ORDER BY total_quantity_sold DESC;
```

🟡 SECTION 2: Intermediate Level (Performance & Distribution)

6. Category Performance & Average Order Value (AOV)

Calculate total revenue and average order value for each product category.

```sql
SELECT
    product_category,
    ROUND(SUM(revenue), 2) AS total_revenue,
    ROUND(AVG(revenue), 2) AS avg_order_value
FROM dbo.ECommerce_Sales_Analytics
GROUP BY product_category
ORDER BY total_revenue DESC;
```

7. Payment Method Distribution

Determine the distribution of orders and total revenue across different payment methods.

```sql
SELECT
    payment_method,
    COUNT(order_id) AS total_orders,
    ROUND(SUM(revenue), 2) AS total_revenue
FROM dbo.ECommerce_Sales_Analytics
GROUP BY payment_method
ORDER BY total_orders DESC;
```

8. High-Revenue Product Categories Filter

Find all product categories that generated a total revenue greater than $1,000,000.

```sql
SELECT
    product_category,
    ROUND(SUM(revenue), 2) AS total_revenue
FROM dbo.ECommerce_Sales_Analytics
GROUP BY product_category
HAVING SUM(revenue) > 1000000
ORDER BY total_revenue DESC;
```

9. Monthly Sales Breakdown

Analyze monthly revenue trends using Date functions (MONTH(), YEAR()).

```sql
SELECT
    YEAR(order_date) AS sales_year,
    MONTH(order_date) AS sales_month,
    ROUND(SUM(revenue), 2) AS monthly_revenue
FROM dbo.ECommerce_Sales_Analytics
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY sales_year, sales_month;
```

10. High-Performing Payment Methods via Subquery Filter

Identify payment methods that have an average order value higher than the overall company average.

```
SELECT
    payment_method,
    ROUND(AVG(revenue), 2) AS avg_payment_aov
FROM dbo.ECommerce_Sales_Analytics
GROUP BY payment_method
HAVING AVG(revenue) > (
    SELECT AVG(revenue)
    FROM dbo.ECommerce_Sales_Analytics
)
ORDER BY avg_payment_aov DESC;
```

🔴 SECTION 3: Advanced Level (Analytical Deep-Dives)

11. Top Product Category Per Region (Window Functions)

Identify the #1 revenue-generating product category within each region using DENSE_RANK().

```
WITH RankedCategories AS (
    SELECT
        region,
        product_category,
        ROUND(SUM(revenue), 2) AS total_revenue,
        DENSE_RANK() OVER (
            PARTITION BY region
            ORDER BY SUM(revenue) DESC
        ) AS rnk
    FROM dbo.ECommerce_Sales_Analytics
    GROUP BY region, product_category
)
SELECT
    region,
    product_category,
    total_revenue
FROM RankedCategories
WHERE rnk = 1
ORDER BY region;
```

12. Percentage Revenue Contribution per Category

Calculate the percentage contribution of each product category to total company revenue.

```
SELECT
    product_category,
    ROUND(SUM(revenue), 2) AS category_revenue,
    ROUND(
        (SUM(revenue) * 100.0) / SUM(SUM(revenue)) OVER(), 2
    ) AS percentage_contribution
FROM dbo.ECommerce_Sales_Analytics
GROUP BY product_category
ORDER BY percentage_contribution DESC;
```

13. Discount Tier Impact on Customer Ratings

Determine the average customer rating for orders grouped by discount percentage brackets.

```sql
SELECT
    discount_category,
    ROUND(AVG(customer_rating), 2) AS avg_rating
FROM (
    SELECT
        customer_rating,
        CASE
            WHEN discount = 0 THEN 'No discount'
            WHEN discount > 0 AND discount <= 0.10 THEN 'Low Discount'
            WHEN discount > 0.10 AND discount <= 0.20 THEN 'Medium discount'
            ELSE 'High Discount'
        END AS discount_category
    FROM dbo.ECommerce_Sales_Analytics
) AS sub
GROUP BY discount_category
ORDER BY avg_rating DESC;
```

14. Monthly Sales Trends & Cumulative Revenue (Running Total)

Calculate monthly revenue along with a month-by-month Cumulative Revenue (Running Total).

```sql
WITH MonthlySales AS (
    SELECT
        YEAR(order_date) AS sales_year,
        MONTH(order_date) AS sales_month,
        SUM(revenue) AS monthly_revenue
    FROM dbo.ECommerce_Sales_Analytics
    GROUP BY YEAR(order_date), MONTH(order_date)
)
SELECT
    sales_year,
    sales_month,
    ROUND(monthly_revenue, 2) AS monthly_revenue,
    ROUND(
        SUM(monthly_revenue) OVER (
            ORDER BY sales_year, sales_month
        ), 2
    ) AS cumulative_revenue
FROM MonthlySales
ORDER BY sales_year, sales_month;
```

💡 Key Business Insights

1. Impact of Heavy Discounts on Brand Perception

Analysis of rating distributions across discount brackets shows that **zero-discount orders maintain higher average customer satisfaction scores (~3.16)** compared to heavily discounted orders (~2.99). Deep discounting does not inherently correlate with higher customer ratings and may signal quality trade-offs or fulfillment issues for discounted items.

2. Regional Revenue Champions

Performing regional partitioning reveals distinct categorical leaders per geographic domain. Marketing and logistics resources should be dynamically allocated toward stocking top-ranking regional categories rather than applying uniform product pushes.

3. Steady Cumulative Growth Trajectory

Evaluating the month-over-month running total highlights predictable sales velocity, allowing supply chain operations to optimize inventory holding costs and anticipate working capital requirements ahead of peak sales cycles.

4. Category Revenue Concentration

Evaluating product category percentage contributions helps identify core revenue drivers (which account for the bulk of earnings) vs. long-tail product lines that may require catalog restructuring or targeted promotion.